

Project Guidelines and Suggestions

1 Guidelines

The purpose of the project is to help you extend what you learned from the course, practice problem solving methods, and get an early start on research and development. This year, we have two types of projects. One is open-end, development-oriented AI agentic project, where we allow *large groups* to complete large projects. The other is the more traditional, research-oriented projects, where we allow groups of up to *two* students.

2 Suggestions on development projects

Here, we would like to do large prototyping development projects that might lead to some startup ventures.

The main idea behind the following three projects are similar: instead of generating code and then verifying its correctness or debugging, we want AI agent to generate the correctness proof and the code hand-by-hand. We will teach the agent the methodology and a step-by-step guide. In theory, a proof is given as an inductive invariant, and must pass the three checks. Our first task is to automatically formalize the code (in Python, RTL, or any coding language) into TLA Next formula. The second task is to make the proof checking automatic. Then we can construct a feedback loop with the agent.

1. **Agentic system for correct Python code generation** Python is a simple language, and most Python programs are just sequential programs. Here, we need to handle complex programs with many

modules. One suggested approach is to first generate correct PlusCal spec with proof, and then gradually refine the spec into Python code.

2. **Agentic system for correct RTL hardware generation** Hardware designs are critical to request full correctness. Hardware are naturally concurrent and non-deterministic. TLA+ are designed to handle both. A good approach should first generate a correct TLA+ spec with proof, and then refine it into more and more detailed spec, and then translate it into Verilog in the end. It is better also assuming that we have a library of submodule IPs available to be reused.
3. **Agentic system for robust Cyber-security** Here we build agent to generate network protocols and programs with security proof.

3 Suggestions on research projects

Here are some suggested project ideas. You are very encouraged to come up with your own ideas, however, you must be able to justify what you propose. I will be happy to discuss them with you or discuss any ideas you have.

1. **Specification and Verification of Security** Security is the robustness of a system under attacks. In this project, we are looking at different systems and applications and study how to formal specify them and their environment so that security can be formally understood and verified.

Specifically, we want to do a couple of case studies to see how TLA+ and TLC are effective in protocol specification and verification. One starting point is Chang and Shmatikov's "Formal Analysis of Authentication in Bluetooth Device Pairing" (http://www.cs.utexas.edu/~shmat/shmat_fcs07.pdf). They used a process calculus to specify the protocol and a resolution-based verifier called ProVerif to verify it. Your job here is to use TLA+ and TLC and to compare the results with them.

You can do the same on other protocols too.

2. **Probabilistic System Specification and Analysis** Many systems exhibit random or probabilistic behaviours. Many system properties, such as anonymity, are enforced via random behaviours. TLA+ and

TLC cannot be used in these tasks. However, there are languages and tools developed for them. PRISM (<http://www.prismmodelchecker.org>) is one of such tools. In this project, you are suggested to first study PRISM, and then extend TLA+ to specify probabilistic systems. You also need to have a tool to verify the specifications. There are two options: you may modify TLC to handle probability or you may translate your modified TLA+ specs into PRISM.

3. **Specification and Verification of Dijkstra’s Self-Stabilization**

Algorithm Dijkstra designed a self-stabilization algorithm for a distributed system. It has the property that no matter what state the system is at, it will eventually converge to a legal state where exact one of the machines is privileged. The original paper can be found at <http://www.cs.utexas.edu/users/EWD/ewd03xx/EWD391.PDF>. You need to specify the algorithm and its correctness conditions by TLA, and then prove its correctness. You are also encouraged to modify the algorithm while still maintain its correctness.

4. **Weak binary semaphore** is a binary semaphore with weak fairness.

Thus, if you have two processes using a single binary semaphore to protect a critical region, it is possible for one process to never enter the critical region: it could infinitely often find itself in contention with the other process, and the other process could always win the contention. An example of a weak binary semaphore is one implemented with “test and set”.

Implement n process mutual exclusion using only weak binary semaphores. The number of processes n is a parameter of the system. Show your solution is correct using TLA+.

5. **Barrier synchronization** is a concurrency construct that allows a (fixed) set of threads to block until all threads have arrived at the barrier. If $barrier(n)$ is the construct, then a thread blocks when it executes $barrier(n)$ until all n threads have executed $barrier(n)$. Then, each blocked thread unblocks, but blocks again if it again executes $barrier(n)$.

First, come up with a lower bound on the number of semaphores that are required to implement barrier synchronization. Your argument can be informal. Then, devise a solution that uses as few semaphores as you can - ideally, the number you showed to be a lower bound.

Use TLA+ to show that your solution does indeed implement barrier synchronization.

6. **Consensus** Consider four processes that communicate by sending messages to each other. You can assume that if a process p sends a message to process q at time t , then q will receive the message by time $t + D$ for some known constant D . Each process i has a value $i.input$. Your goal is to design a protocol such that each correct process i determines a value $i.output$ where

- For any two nonfaulty processes i and j , $i.output = j.output$.
- If, for all processes i , $i.input = v$, then each nonfaulty process j sets $j.output$ to v .

No more than one process can be faulty. A faulty process can send any message it wishes. For example, it can tell one correct process that its input value was 1 and tell another process that its input value was 17. It can also fail to send some messages, or simply stop running. Show that your protocol is correct using TLA+.

7. **Cache Coherence** For today's multicore processors, cache coherence protocols are very important for operation correctness. Chapter 5 in the TLA+ book has covered some basics on cache protocols. You can consider to specify and verify many wild protocols.

4 Submissions and deadlines

Apr 12: Reach a decision on which project you want to do. Submit the title, a brief description of the ideas and approaches.

May 1: A midterm progress report. Have you achieved substantial progress?

Jun 5: A complete report with all theories, results, and conclusions.